



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 物理内存与虚拟内存管理

专业名称: 计算机科学与技术

学生姓名: 林宏宇

学生学号: 23320093

实验地点: 实验中心 D501

实验时间: 2025 年 5 月 30 日

Section 1 实验概述

在本次实验中，我们首先学习如何使用位图和地址池来管理资源。然后，我们将实现在物理地址空间下的内存管理。接着，我们将会学习并开启二级分页机制。在开启分页机制后，我们将实现在虚拟地址空间下的内存管理。

本次实验最精彩的地方在于分页机制。基于分页机制，我们可以将连续的虚拟地址空间映射到不连续的物理地址空间。同时，对于同一个虚拟地址，在不同的页目录表和页表下，我们会得到不同的物理地址。这为实现虚拟地址空间的隔离奠定了基础。但是，本实验最令人困惑的地方也在于分页机制。开启了分页机制后，程序中使用的地址是虚拟地址。我们需要结合页目录表和页表才能确定虚拟地址对应的物理地址。而我们常常会忘记这一点，导致了我們不知道某些虚拟地址表示的具体含义。

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具
- 实验环境：
 - 虚拟机版本/处理器型号：Virtualbox
 - 代码编辑环境：Vscode+nasm+C/C++插件
 - 代码编译工具：gcc/g++（64 位）
 - 重要三方库信息：

Section 3 实验任务

- 实验任务 1：物理页内存管理的实现
- 实验任务 2：二级分页机制的实现
- 实验任务 3：虚拟页内存管理的实现
- 实验任务 4：页面置换算法的实现

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：物理页内存管理的实现

复现实验 7 指导书中“物理页内存管理”一节的代码，实现物理页内存的管理，具体要求如下：

1. 结合代码分析位图，地址池，物理页管理的初始化过程，以及物理页进行分配和释放的实现思路
2. 构造测试用例来分析物理页内存管理的实现是否存在 bug。

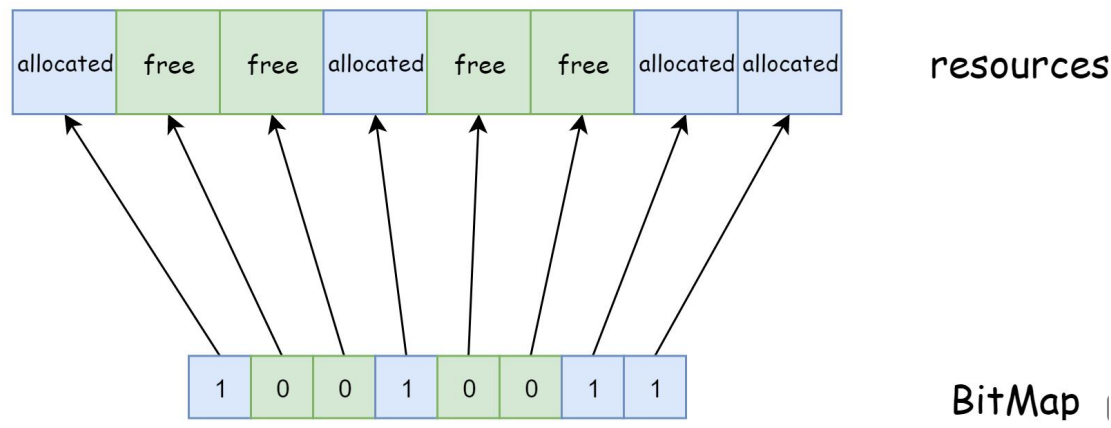
- 实验步骤：

- 1 结合代码分析位图，地址池，物理页管理的初始化过程，以及物理页进行分配和释放的实现思路。

1.1 位图的初始化过程

1.1.1 概念

BitMap 使用一位来和一个资源单元建立映射关系，例如，我们可以使用 1 表示资源单元已经被分配，使用 0 表示资源单元空闲



1.1.2 位图定义代码

BitMap 类的成员包括：一块存放 BitMap 的内存区域、BitMap 管理的资源单元数量、单独存取位和批处理存取位的方法。

```
class BitMap
{
public:
    // 被管理的资源个数, bitmap 的总位数
    int length;

    // bitmap 的起始地址
    char *bitmap;

public:
    // 初始化
    BitMap();

    // 设置 BitMap, bitmap=起始地址, length=总位数(被管理的资源个数)
    void initialize(char *bitmap, const int length);

    // 获取第 index 个资源的状态, true=allocated, false=free
    bool get(const int index) const;

    // 设置第 index 个资源的状态, true=allocated, false=free
    void set(const int index, const bool status);

    // 分配 count 个连续的资源, 若没有则返回 -1, 否则返回分配的第 1 个资源单元序号
    int allocate(const int count);

    // 释放第 index 个资源开始的 count 个资源
    void release(const int index, const int count);

    // 返回 Bitmap 存储区域
    char *getBitmap();

    // 返回 Bitmap 的大小
    int size() const;

private:
    // 禁止 Bitmap 之间的赋值
    BitMap(const BitMap &) {}

    void operator=(const BitMap&) {}
};
```

1.1.3 初始化代码（依据题目要求重点阐明）

一开始初始化时，没有内存资源单元，位图不需要指向，则位图的长度为 0 指针为空。

```
BitMap::BitMap()
{
    initialize(nullptr, 0);
}

void BitMap::initialize(char *bitmap, const int length)
{
    this->bitmap = bitmap;
    this->length = length;

    int bytes = ceil(length, 8);
    memset(bitmap, 0, bytes);
}
```

1.2 地址池的初始化过程

1.2.1 概念

为了实现页内存分配，我们需要使用一种结构来标识地址空间中的哪些页是已经被分配的，哪些是未被分配的。

这种结构被称为地址池。当需要页内存分配时，我们可以从地址池中取出一个空闲页。然后地址池便会标识该空闲页已被分配，最后计算并返回该空闲页对应的地址。空闲页只要没有被释放，就不会被再次分配。

注意到 BitMap 可用于资源的管理，地址池实际上使用了 BitMap 来对其地址空间进行管理。

1.2.2 地址池定义代码

```
class AddressPool
{
public:
    BitMap resources;
    int startAddress;
public:
    AddressPool();
    // 初始化地址池
    void initialize(char *bitmap, const int length, const int startAddress);
    // 从地址池中分配 count 个连续页，成功则返回第一个页的地址，失败则返回-1
    int allocate(const int count);
    // 释放若干页的空间
    void release(const int address, const int amount);
};
```

1.2.3 地址池初始化代码

使用 resources 来标识地址池中哪些页是空闲的，哪些页是被分配的。

使用 startAddress 来记录地址池管理的页的共同起始地址。

```
void AddressPool::initialize(char *bitmap, const int length, const int startAddress)
{
    resources.initialize(bitmap, length);
    this->startAddress = startAddress;
}
```

1.3 物理页管理的初始化过程

1.3.1 概念

我们将整个物理地址空间划分为两部分，内核空间和用户空间。内核需要的物理页只会从内核空间中分配，用户程序需要的物理页也只会从用户空间中分配。

在我们的操作系统实现当中，我们会使用两个地址池来对这两部分物理地址进行管理。

1.3.2 使用类 MemoryManager 来执行内存管理

```
class MemoryManager
{
public:
    // 可管理的内存容量
    int totalMemory;

    // 内核物理地址池
    AddressPool kernelPhysical;

    // 用户物理地址池
    AddressPool userPhysical;

public:
    MemoryManager();

    // 初始化地址池
    void initialize();

    // 从 type 类型的物理地址池中分配 count 个连续的页
    // 成功，返回起始地址；失败，返回 0
    int allocatePhysicalPages(enum AddressPoolType type, const int count);

    // 释放从 paddr 开始的 count 个物理页
    void releasePhysicalPages(enum AddressPoolType type, const int paddr, const int count);

    // 获取内存总容量
    int getTotalMemory();
};
```

1.3.3 物理页管理的初始化过程（依据题目要求重点阐明）

先预留部分内存，然后将剩下的内存对半分给内核空间和用户空间，然后分别设置起始地址，初始化用户空间和内核空间的 address_pool。

```
MemoryManager::MemoryManager() {
    initialize();
}

void MemoryManager::initialize()
{
    this->totalMemory = 0;
    this->totalMemory = getTotalMemory();

    // 预留的内存
    int usedMemory = 256 * PAGE_SIZE + 0x100000;
    if(this->totalMemory < usedMemory) {
        printf("memory is too small, halt.\n");
        asm_halt();
    }

    // 剩余的空闲的内存
    int freeMemory = this->totalMemory - usedMemory;
```

```

int freePages = freeMemory / PAGE_SIZE;
int kernelPages = freePages / 2;
int userPages = freePages - kernelPages;
int kernelPhysicalStartAddress = usedMemory;
int userPhysicalStartAddress = usedMemory + kernelPages * PAGE_SIZE;
int kernelPhysicalBitMapStart = BITMAP_START_ADDRESS;
int userPhysicalBitMapStart = kernelPhysicalBitMapStart + ceil(kernelPages, 8);
kernelPhysical.initialize((char *)kernelPhysicalBitMapStart, kernelPages, kernelPhysicalStartAddress);
userPhysical.initialize((char *)userPhysicalBitMapStart, userPages, userPhysicalStartAddress);
printf("total memory: %d bytes ( %d MB )\n",
       this->totalMemory,
       this->totalMemory / 1024 / 1024);
printf("kernel pool\n"
       "  start address: 0x%x\n"
       "  total pages: %d ( %d MB )\n"
       "  bitmap start address: 0x%x\n",
       kernelPhysicalStartAddress,
       kernelPages, kernelPages * PAGE_SIZE / 1024 / 1024,
       kernelPhysicalBitMapStart);
printf("user pool\n"
       "  start address: 0x%x\n"
       "  total pages: %d ( %d MB )\n"
       "  bit map start address: 0x%x\n",
       userPhysicalStartAddress,
       userPages, userPages * PAGE_SIZE / 1024 / 1024,
       userPhysicalBitMapStart);
}

```

1.4 物理页的分配和释放

1.4.1 物理页的分配

```
int MemoryManager::allocatePhysicalPages(enum AddressPoolType type, const int count)
{
    int start = -1;
    if (type == AddressPoolType::KERNEL)
    {
        start = kernelPhysical.allocate(count);
    }
    else if (type == AddressPoolType::USER)
    {
        start = userPhysical.allocate(count);
    }
    return (start == -1) ? 0 : start;
}
```

1.4.2 物理页的释放

```
void MemoryManager::releasePhysicalPages(enum AddressPoolType type, const int paddr, const int count)
{
    if (type == AddressPoolType::KERNEL)
    {
        kernelPhysical.release(paddr, count);
    }
    else if (type == AddressPoolType::USER)
    {
        userPhysical.release(paddr, count);
    }
}
```

2. 构造测试用例来分析物理页内存管理的实现是否存在 bug。如果存在，则尝试修复并再次测试。否则，结合测试用例简要分析物理页内存管理的实现的正确性。

2.1 测试用例

测试例子代码

```
void first_thread(void *arg)
{
    printf("-----MemoryManager-----Lhy23320093-----\n");
    int start_add_1 = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 1);
    int start_add_2 = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 2);
    int start_add_3 = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 3);
    memoryManager.releasePhysicalPages(AddressPoolType::USER, start_add_3, 3);
    memoryManager.releasePhysicalPages(AddressPoolType::USER, start_add_2, 2);
    int start_add_4 = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 4);
    memoryManager.releasePhysicalPages(AddressPoolType::USER, start_add_4, 4);
    memoryManager.releasePhysicalPages(AddressPoolType::USER, start_add_3, 3);
    asm_halt();
}
```

辅助观察输出代码，在 memory.cpp 文件的分配和释放函数下

```
if(start != -1)
    printf("Allocate %d page ; Start_address = %x\n", count, start);

printf("Release %d page ; Start_address = %x\n", count, paddr);
```

2.2 测试结果

```
-----MemoryManager-----Lhy23320093-----
Allocate 1 page ; Start_address = 4070000
Allocate 2 page ; Start_address = 4071000
Allocate 3 page ; Start_address = 4073000
Release 3 page ; Start_address = 4073000
Release 2 page ; Start_address = 4071000
Allocate 4 page ; Start_address = 4071000
Release 4 page ; Start_address = 4071000
Release 3 page ; Start_address = 4073000
```

2.3 结合用例，分析内存管理的实现

第一次分配，内存在 0x4070000 处分配一个页。

第二次分配，内存在 0x4071000 处分配两个页。

第三次分配，内存在 0x4073000 处分配三个页。

第一次释放，释放第三个进程的三个页。

第二次释放，释放第二个进程的两个页。

第四次分配，内存在 0x4071000 处分配四个页。（前面占用的地址释放后又空出来了）

第三次释放，释放第四个进程的四个页。

第四次释放，释放第一个进程的一个页。

- 实验结果展示：结果已经逐步体现在上述的实验过程中了。

----- 实验任务 2 -----

- 任务要求：二级分页机制的实现

复现实验 7 指导书中“二级分页机制”一节的代码，实现二级分页机制，具体要求如下：

1. 实现内存的申请和释放，保存实验截图并对能够在虚拟地址空间中进行内存管理，截图并给出过程解释。

2. 相比于一级页表，二级页表的开销是增大了的，但操作系统中往往使用的是二级页表而不是一级页表。结合你自己的实验过程，说说相比于一级页表，使用二级页表会带来哪些优势。

- 实验步骤：

1. 实现分页机制

1.1 在内存管理器 MemoryManager 加入开启分页机制的成员函数声明

```
class MemoryManager
{
    ...
    // 开启分页机制
    void openPageMechanism();
};
```

1.2 开启分页机制的函数定义

```
void MemoryManager::openPageMechanism()
{
    // 页目录表指针
    int *directory = (int *)PAGE_DIRECTORY;

    // 线性地址 0~4MB 对应的页表
    int *page = (int *)(PAGE_DIRECTORY + PAGE_SIZE);

    // 初始化页目录表
    memset(directory, 0, PAGE_SIZE);

    // 初始化线性地址 0~4MB 对应的页表
    memset(page, 0, PAGE_SIZE);

    int address = 0;

    // 将线性地址 0~1MB 恒等映射到物理地址 0~1MB
    for (int i = 0; i < 256; ++i)
    {
        // U/S = 1, R/W = 1, P = 1
        page[i] = address | 0x7;
        address += PAGE_SIZE;
    }

    // 初始化页目录项
    // 0~1MB
    directory[0] = ((int)page) | 0x07;

    // 3GB 的内核空间
    directory[768] = directory[0];

    // 最后一个页目录项指向页目录表
    directory[1023] = ((int)directory) | 0x7;

    // 初始化 cr3, cr0, 开启分页机制
    asm_init_page_reg(directory);

    printf("open page mechanism\n");
}
```

1.3 开启分页机制

```
extern "C" void setup_kernel()
{
    ...

    // 内存管理器
    memoryManager.openPageMechanism();
    memoryManager.initialize();

    // 创建第一个线程
    ...
}
```

1.4 申请和释放资源进程代码

进程代码

分配 p1: 在 kernel 分配内存 100 页

分配 p2: 在 kernel 分配内存 10 页

分配 p3: 在 kernel 分配内存 100 页

释放 p2

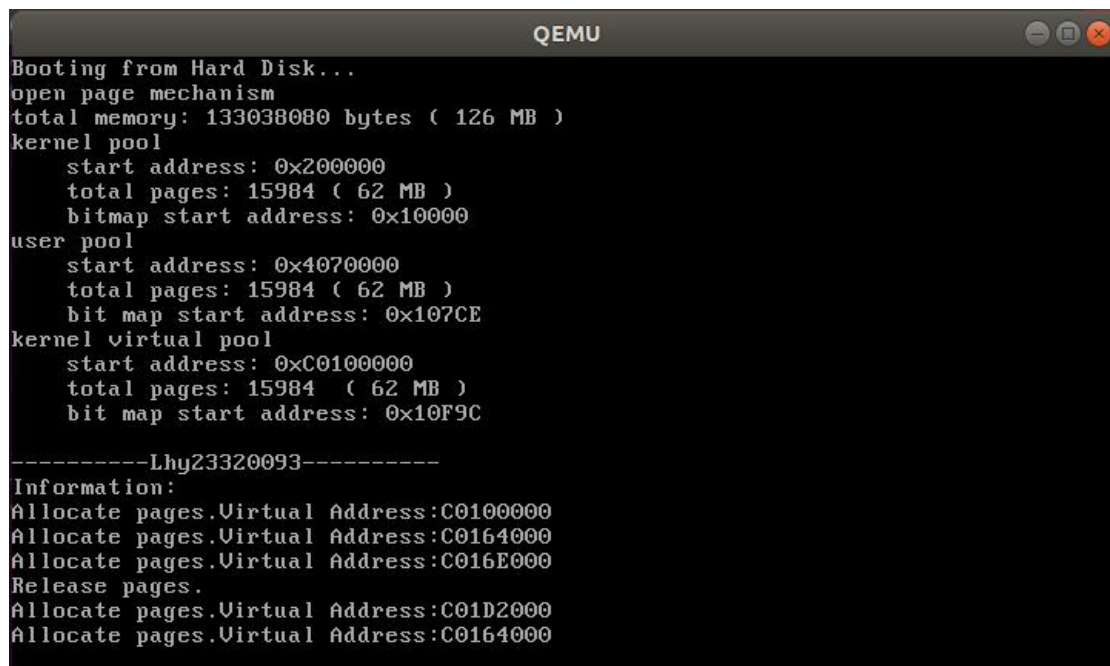
分配 p2: 在 kernel 分配内存 100 页

再分配 p2: 在 kernel 分配内存 10 页

```
void first_thread(void *arg)
{
    printf("\n-----Lhy23320093-----\n");
    printf("Information:\n");
    char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);
    char *p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
    char *p3 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);
    memoryManager.releasePages(AddressPoolType::KERNEL, (int)p2, 10);
    p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);
    p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);

    asm_halt();
}
```

1.5 结果



```
QEMU
Booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
    start address: 0x200000
    total pages: 15984 ( 62 MB )
    bitmap start address: 0x10000
user pool
    start address: 0x4070000
    total pages: 15984 ( 62 MB )
    bit map start address: 0x107CE
kernel virtual pool
    start address: 0xC0100000
    total pages: 15984 ( 62 MB )
    bit map start address: 0x10F9C
-----Lhy23320093-----
Information:
Allocate pages.Virtual Address:C0100000
Allocate pages.Virtual Address:C0164000
Allocate pages.Virtual Address:C016E000
Release pages.
Allocate pages.Virtual Address:C01D2000
Allocate pages.Virtual Address:C0164000
```

在 memory 的内存分配和释放中添加了辅助观察虚拟地址和物理地址的代码。

观察虚拟地址空间：p2 释放掉之后又申请了一个 100 的空间，p2 释放完之后原地只剩了 10 的空间，因此分配时一定是在 p3 之后。

再给 p2 申请了 10 页的空间，刚好等于之前 p1 和 p3 之间释放掉的 10 页空间，因此可以重新给 p2 分配，虚拟地址又与第一次 p2 的虚拟地址相等。

2. 相比于一级页表，二级页表的开销是增大了的，但操作系统中往往使用的是二级页表而不是一级页表。结合你自己的实验过程，说说相比于一级页表，使用二级页表会带来哪些优势。

2.1. 动态分配与内存开销优化

一级页表的问题：

需要为整个虚拟地址空间（如 32 位的 4GB）预先分配完整的页表（1M 项 × 4B=4MB），即使大部分虚拟地址并未使用，导致内存浪费。

进程增多时，多个 4MB 的页表会占用大量物理内存。

二级页表的优势：

按需分配页表：仅当虚拟地址空间的实际区域被使用时，才分配对应的页表和页目录项。例如，实验中仅映射了内核所在的 0~1MB 区域，其他未使用的页表无需分配。

节省内存：进程的页目录表（固定 4KB）和少量实际使用的页表（如实验中仅 1 个 4KB 页表），显著减少内存占用。在多进程环境下，此优势更为突出。

2.2. 灵活的地址映射与动态重定位

实验中的动态重定位需求：

内核被加载到物理地址 0x20000 处，但编译时默认起始地址为 0。通过 ld -Ttext 0x00020000 调整链接地址，实现绝对装入。若加载位置变化，需重新链接。

二级页表的动态映射：

虚拟地址到物理地址的灵活映射：通过修改页目录项和页表项，同一虚拟地址可映射到不同物理地址，无需重新链接程序。例如：

内核虚拟地址 0x80000000 可通过页表映射到物理地址 0x20000。

进程的虚拟地址空间独立，均从 0 开始，但通过各自的页表映射到不同物理内存，实现进程隔离。

支持程序动态加载：进程的物理内存分配无需连续，通过页表离散映射，避免内存碎片问题。

2.3. 内存保护机制

页表项的权限控制：

通过页目录项和页表项的 U/S（用户/内核权限）、R/W（读写权限）等标志位，实现内存保护。例如：

内核页表项设置 U/S=0，禁止用户态进程访问内核空间。

只读代码段的页表项设置 R/W=0，防止意外修改。

进程隔离：每个进程拥有独立的页目录表，确保其虚拟地址空间无法访问其他进程的物理内存。实验中的多进程/线程依赖此机制实现互不干扰。

2.4. 共享内存与高效内存管理

共享物理页：

不同进程的页表项可指向同一物理页，实现共享内存（如共享库、进程间通信）。

例如：

内核代码只需加载到物理内存一次，所有进程通过页表映射共享同一份只读副本。

延迟分配与按需分页：

页表项可标记为“不存在”（P=0），访问时触发缺页异常，由操作系统动态分配物理页并填充页表项。此机制支持动态内存扩展和交换空间管理。

实验中的具体体现

内核地址空间映射：

实验中通过 `directory[0]` 映射 0~1MB 的物理内存（恒等映射），同时通过 `directory[768]` 将同一物理区域映射到内核虚拟地址 `0xC0000000`（3GB），实现内核代码的高半地址运行。

优势：用户程序使用低地址空间（0~3GB），内核使用高地址空间（3~4GB），通过页目录项权限隔离，避免用户程序越界访问内核。

页目录自映射：

`directory[1023]` 指向页目录表自身，使得通过虚拟地址 `0xFFFFF000` 可直接访问页目录表，简化页表修改操作（如实验中调整映射关系）。

2.5 总结

二级页表通过层次化结构（页目录表→页表→物理页）实现了按需分配内存、动态地址映射、细粒度权限控制和进程隔离，有效解决了一级页表的内存浪费问题。结合实验中的多进程、内核隔离和动态加载需求，二级页表在保证性能的同时，提供了灵活高效的内存管理机制。

- 实验结果展示：实验结果在过程中已经体现。

----- 实验任务 3 -----

- 任务要求：虚拟页内存管理的实现

复现实验 7 指导书中“虚拟页内存管理”一节的代码，实现虚拟页内存的管理，具体要求如下：

1. 结合代码，描述虚拟页内存分配的三个基本步骤，以及虚拟页内存的释放的过程。

2. 构造测试用例来分析虚拟页内存管理的实现是否存在 bug，如果存在，则尝试修复并再次测试。否则，结合测试用例简要分析虚拟页内存管理的实现的正确性。

3. 在 PDE(页目录项)和 PTE(页表项)的虚拟地址构造中，我们使用了第 1023 个页目录项。第 1023 个页目录项指向了页目录表本身，从而使得我们可以构造出 PDE 和 PTE 的虚拟地址。现在，我们将这个指向页目录表本身的页目录项放入第 1000 个页目录项，而不再是放入了第 1023 个页目录项。请同学们借助第 1000 个页目录项，构造出第 141 个页目录项的虚拟地址，和第 891 个页目录项指向的页表中第 109 个页表项的虚拟地址。

- 实验步骤：

1. 结合代码，描述虚拟页内存分配的三个基本步骤，以及虚拟页内存的释放的过程。

1.1 虚拟页内存分配的三个基本步骤

第一步，从虚拟地址池中分配若干连续的虚拟页

第二步，对每一个虚拟页，从物理地址池中分配 1 页

第三步，为虚拟页建立页目录项和页表项，使虚拟页内的地址经过分页机制变换到物理页内

```

int MemoryManager::allocatePages(enum AddressPoolType type, const int count)
{
    // 第一步：从虚拟地址池中分配若干虚拟页
    int virtualAddress = allocateVirtualPages(type, count);
    if (!virtualAddress)
    {
        return 0;
    }

    bool flag;
    int physicalPageAddress;
    int vaddress = virtualAddress;
    // 依次为每一个虚拟页指定物理页
    for (int i = 0; i < count; ++i, vaddress += PAGE_SIZE)
    {
        flag = false;
        // 第二步：从物理地址池中分配一个物理页
        physicalPageAddress = allocatePhysicalPages(type, 1);
        if (physicalPageAddress)
        {
            //printf("allocate physical page 0x%x\n", physicalPageAddress);

            // 第三步：为虚拟页建立页目录项和页表项，使虚拟页内的地址经过分页机制变换到物理页内。
            flag = connectPhysicalVirtualPage(vaddress, physicalPageAddress);
        }
        else
        {
            flag = false;
        }
        // 分配失败，释放前面已经分配的虚拟页和物理页表
        if (!flag)
        {
            // 前 i 个页表已经指定了物理页
            releasePages(type, virtualAddress, i);
            // 剩余的页表未指定物理页
            releaseVirtualPages(type, virtualAddress + i * PAGE_SIZE, count - i);
            return 0;
        }
    }
    return virtualAddress;
}

```

1.2 虚拟页内存的释放过程

页内存的释放是页内存分配的逆过程，分两个步骤完成。

第一步，对每一个虚拟页，释放为其分配的物理页。

```
int MemoryManager::vaddr2paddr(int vaddr)
{
    int *pte = (int *)toPTE(vaddr);
    int page = (*pte) & 0xfffff000;
    int offset = vaddr & 0xfff;
    return (page + offset);
}
```

第二步，释放虚拟页。

```
void MemoryManager::releaseVirtualPages(enum AddressPoolType type, const int vaddr, const int count)
{
    if (type == AddressPoolType::KERNEL)
    {
        kernelVirtual.release(vaddr, count);
    }
}
```

2. 构造测试用例来分析虚拟页内存管理的实现是否存在 bug，如果存在，则尝试修复并再次测试。否则，结合测试用例简要分析虚拟页内存管理的实现的正确性。

2.1 运行样例代码

```
void first_thread(void *arg)
{
    printf("\n-----Lhy23320093-----\n");
    printf("Information:\n");
    char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 3);
    char *p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 4);
    char *p3 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 2);
    memoryManager.releasePages(AddressPoolType::KERNEL, (int)p2, 4);
    p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 1);
    asm_halt();
}
```

2.2 运行结果

```
-----Lhy23320093-----
Information:
Allocate 1 pages.PhysicalAddress_Start = 200000
Allocate 1 pages.PhysicalAddress_Start = 201000
Allocate 1 pages.PhysicalAddress_Start = 202000
Allocate pages.Virtual Address:C0100000 Physical Address:202000
Allocate 1 pages.PhysicalAddress_Start = 203000
Allocate 1 pages.PhysicalAddress_Start = 204000
Allocate 1 pages.PhysicalAddress_Start = 205000
Allocate 1 pages.PhysicalAddress_Start = 206000
Allocate pages.Virtual Address:C0103000 Physical Address:206000
Allocate 1 pages.PhysicalAddress_Start = 207000
Allocate 1 pages.PhysicalAddress_Start = 208000
Allocate pages.Virtual Address:C0107000 Physical Address:208000
Release 1 pages.PhysicalAddress_Start = 203000
Release 1 pages.PhysicalAddress_Start = 204000
Release 1 pages.PhysicalAddress_Start = 205000
Release 1 pages.PhysicalAddress_Start = 206000
Release pages.
Allocate 1 pages.PhysicalAddress_Start = 203000
Allocate pages.Virtual Address:C0103000 Physical Address:203000
```

根据输出的结果，可以观察到每次分配和释放都在正确的地址空间。

其中在最后两行输出，释放 p2 后，再给 p2 分配空间。

无论是虚拟地址还是物理地址都能重新利用之前被释放的空间（虚拟地址 C0103000、物理地址 203000 之前已经出现过然后释放了）

3. 将这个指向页目录表本身的页目录项放入第 1000 个页目录项，而不再是放入了第 1023 个页目录项。请同学们借助第 1000 个页目录项，构造出第 141 个页目录项的虚拟地址，和第 891 个页目录项指向的页表中第 109 个页表项的虚拟地址

3.1 在代码中的体现

在 `void MemoryManager::openPageMechanism()` 函数下

将指向页目录表本身的页目录项放入第 1000 个页目录项

```
// 最后一个页目录项指向页目录表
directory[1000] = ((int)directory) | 0x7;
```

3.2 虚拟地址的组成部分

页目录索引 11 位+页表索引 10 位+页内偏移 12 位

3.4 构造第 141 个页目录项的虚拟地址

要访问第 i 个页目录项 ($PDE[i]$)

$virtual_addr = (1000 \ll 22) + (i \ll 2)$

即 0xFA000234

3.5 第 891 个页目录项指向的页表中第 109 个页表项的虚拟地址
要访问第 i 个页目录项指向的页表中的第 j 个页表项 ($PTE[j]$ in $PT[i]$)

$virtual_addr = (1000 \ll 22) + (i \ll 12) + (j \ll 2)$

即 0xFA37B1B4

- 实验结果展示：代码已经在过程中体现，不再赘述。

----- 实验任务 4 -----

- 任务要求：页面置换算法的实现

(选做内容，如果完成，可附加实验完成度评分) 在 Assignment 3 的基础上，实现一种理论课上学习到的虚拟内存管理中的页面置换算法，在虚拟页内存中实现页面的置换，比如下面所列算法的其中一种：

先进先出页面置换(FIFO)

最优页面置换(OPR)

最近最少使用页面置换(LRU)

最不经常使用页面置换(LFU)

上述置换算法的细节参见理论课教材(《操作系统概念》，原书第 9 版，中文)第 272-280 页，你也可以实现一种自己设计的置换算法。要求:描述你的设计思路并展示页面置换结果的截图(也可以统计缺页错误发生的次数作为输出)。

- 实验步骤：

1. 原理说明

本程序设定系统具有 3 个物理页面帧，分别用 $p1$ 、 $p2$ 和 $p3$ 表示。通过一个整型数组 `pagelist` 模拟页面访问序列，程序依次访问其中的每一个页面。用 `plist` 数组表示当前驻留在内存中的三个页面，每次访问时首先检查该页是否已存在于 `plist` 中。如果命中，表示该页已在内存中，无需置换；否则发生缺页 (MISS)，此时按照 FIFO 策略，替换最早加载的页面。为了记录最早进入内存的页面位置，程序使用 `first` 指针循环递增，实现页面帧的轮转更新。

2. 代码展示

```
void first_thread(void *arg)
{
    wait_for_a_while();

    printf("\n===== Page Replacement Simulation Start =====\n");

    char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);
    char *p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
    char *p3 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);
    printf("Allocated Page Frames:\n");
    printf("  Frame 1: 0x%-10x\n", p1);
    printf("  Frame 2: 0x%-10x\n", p2);
    printf("  Frame 3: 0x%-10x\n", p3);
    wait_for_a_while();
    int pagelist[8] = {7, 3, 3, 7, 2, 1, 9, 8};
    int plist[3] = {0, 0, 0};
    int first = 0;
    int miss = 0;
    int hit = 0;
    int got = -1;
    char *pp;
    printf("\nChecking Page Access Sequence:\n");
    for (int i = 0; i < 8; i++) {
        printf("-----\n");
        printf("Accessing Page: %d\n", pagelist[i]);
        int flag = 0;
        for (int j = 0; j < 3; j++) {
            if (plist[j] == pagelist[i]) {
                flag = 1;
                got = j;
            }
        }
        if (flag == 0) {
            printf("  Result      : MISS (Page Fault)\n");
            plist[first] = pagelist[i];
            printf("  Replaced   : Frame %d <- Page %d\n", first + 1, pagelist[i]);
            printf("  Frame State: [0x%x=%d] [0x%x=%d] [0x%x=%d]\n",
                p1, plist[0], p2, plist[1], p3, plist[2]);
            first = (first + 1) % 3;
            miss++;
        } else {
            if (got == 0) pp = p1;
            if (got == 1) pp = p2;
            if (got == 2) pp = p3;
```

```

        printf("  Result      : HIT (Found in Frame %d)\n", got + 1);

        printf("  Frame Addr : 0x%x\n", pp);

        hit++;

    }

    wait_for_a_while(); //每次处理完一页停顿一下

}

printf("=====\n");

printf("Simulation Summary:\n");

printf("  Total Accesses: %d\n", 8);

printf("  Page Misses   : %d\n", miss);

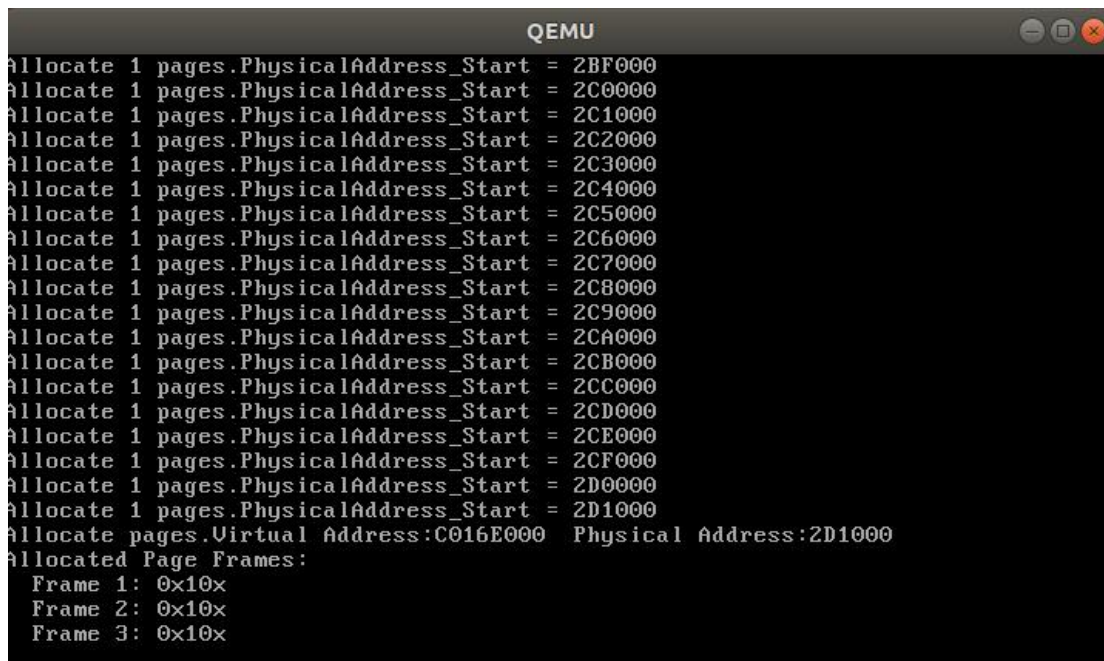
printf("  Page Hits     : %d\n", hit);

printf("=====\n");

asm_halt();
}

```

3. 输出结果展示



The screenshot shows a QEMU terminal window with the following output:

```

QEMU
Allocate 1 pages.PhysicalAddress_Start = 2BF000
Allocate 1 pages.PhysicalAddress_Start = 2C0000
Allocate 1 pages.PhysicalAddress_Start = 2C1000
Allocate 1 pages.PhysicalAddress_Start = 2C2000
Allocate 1 pages.PhysicalAddress_Start = 2C3000
Allocate 1 pages.PhysicalAddress_Start = 2C4000
Allocate 1 pages.PhysicalAddress_Start = 2C5000
Allocate 1 pages.PhysicalAddress_Start = 2C6000
Allocate 1 pages.PhysicalAddress_Start = 2C7000
Allocate 1 pages.PhysicalAddress_Start = 2C8000
Allocate 1 pages.PhysicalAddress_Start = 2C9000
Allocate 1 pages.PhysicalAddress_Start = 2CA000
Allocate 1 pages.PhysicalAddress_Start = 2CB000
Allocate 1 pages.PhysicalAddress_Start = 2CC000
Allocate 1 pages.PhysicalAddress_Start = 2CD000
Allocate 1 pages.PhysicalAddress_Start = 2CE000
Allocate 1 pages.PhysicalAddress_Start = 2CF000
Allocate 1 pages.PhysicalAddress_Start = 2D0000
Allocate 1 pages.PhysicalAddress_Start = 2D1000
Allocate pages.Virtual Address:C016E000 Physical Address:2D1000
Allocated Page Frames:
  Frame 1: 0x10x
  Frame 2: 0x10x
  Frame 3: 0x10x

```

```

QEMU
Allocate 1 pages.PhysicalAddress_Start = 2CF000
Allocate 1 pages.PhysicalAddress_Start = 2D0000
Allocate 1 pages.PhysicalAddress_Start = 2D1000
Allocate pages.Virtual Address:C016E000 Physical Address:2D1000
Allocated Page Frames:
  Frame 1: 0x10x
  Frame 2: 0x10x
  Frame 3: 0x10x

Checking Page Access Sequence:
-----
Accessing Page: 7
  Result      : MISS (Page Fault)
  Replaced    : Frame 1 <- Page 7
  Frame State: [0xC0100000=7] [0xC0164000=0] [0xC016E000=0]
-----
Accessing Page: 3
  Result      : MISS (Page Fault)
  Replaced    : Frame 2 <- Page 3
  Frame State: [0xC0100000=7] [0xC0164000=3] [0xC016E000=0]
-----
Accessing Page: 3
  Result      : HIT (Found in Frame 2)
  Frame Addr  : 0xC0164000

```

```

QEMU
  Frame State: [0xC0100000=7] [0xC0164000=0] [0xC016E000=0]
-----
Accessing Page: 3
  Result      : MISS (Page Fault)
  Replaced    : Frame 2 <- Page 3
  Frame State: [0xC0100000=7] [0xC0164000=3] [0xC016E000=0]
-----
Accessing Page: 3
  Result      : HIT (Found in Frame 2)
  Frame Addr  : 0xC0164000
-----
Accessing Page: 7
  Result      : HIT (Found in Frame 1)
  Frame Addr  : 0xC0100000
-----
Accessing Page: 2
  Result      : MISS (Page Fault)
  Replaced    : Frame 3 <- Page 2
  Frame State: [0xC0100000=7] [0xC0164000=3] [0xC016E000=2]
-----
Accessing Page: 1
  Result      : MISS (Page Fault)
  Replaced    : Frame 1 <- Page 1
  Frame State: [0xC0100000=1] [0xC0164000=3] [0xC016E000=2]

```

```
QEMU
Result      : MISS (Page Fault)
Replaced    : Frame 3 <- Page 2
Frame State: [0xC0100000=7] [0xC0164000=3] [0xC016E000=2]
-----
Accessing Page: 1
Result      : MISS (Page Fault)
Replaced    : Frame 1 <- Page 1
Frame State: [0xC0100000=1] [0xC0164000=3] [0xC016E000=2]
-----
Accessing Page: 9
Result      : MISS (Page Fault)
Replaced    : Frame 2 <- Page 9
Frame State: [0xC0100000=1] [0xC0164000=9] [0xC016E000=2]
-----
Accessing Page: 8
Result      : MISS (Page Fault)
Replaced    : Frame 3 <- Page 8
Frame State: [0xC0100000=1] [0xC0164000=9] [0xC016E000=8]
=====
Simulation Summary:
Total Accesses: 8
Page Misses    : 6
Page Hits      : 2
=====
```

从 QEMU 可以看见每次 Push 和 Pop 的过程。

总访问次数：8

MISS 次数（缺页）：6

HIT 次数（命中）：2

算法性能还算可以

- 实验结果展示：代码在步骤过程中展示，不再赘述。

Section 5 实验总结与心得体会

1. 通过本次关于“物理内存与虚拟内存管理”的操作系统实验，我对现代操作系统中内存管理的基本机制有了更深入的理解，尤其是在理论与代码实现相结合的过程中，加深了对位图管理、地址池划分、二级分页机制和页面分配策略等关键技术点的掌握。

2. 在实验初期，通过对 BitMap 和 AddressPool 的实现与测试，我掌握了物理页内存管理的基本原理和实现方式，认识到如何用有限的物理内存实现对资源的高效分配与释放。随后在分页机制的实现中，体会到虚拟地址空间与物理地址空间之间的转换关系，特别是二级页表的建立逻辑，为后续理解多进程内存隔离与保护机制提供了坚实基础。

3. 本次实验中最具挑战性的部分在于分页机制的初始化与调试，尤其在切换至虚拟地址后，常常因为忽略地址变换而导致调试困难。但正是这些问题让我更加深刻地意识到，操作系统底层的每一步实现都高度依赖准确的内存映射和数据结构设计，容不得丝毫疏忽。

4. 总的来说，本实验不仅锻炼了我的系统思维与代码实现能力，也增强了我对操作系统核心机制的理解，特别是从“逻辑地址”到“物理地址”的转换过程，有助于我在今后的系统编程中打下坚实的基础。